

CS319 — OBJECT-ORIENTED SOFTWARE ENGINEERING

Deliverable 4

High-Level Architecture

Bilsen Review System

Team: **TeamNo**

Date: **April 18, 2026**

Submission Deadline: April 19, 2026 — 23:59

1. Design Goals

The Bilsen Review System replaces BILSEN's current review workflow — coordinated through email threads and Overleaf comments — with a centralized web application that manages paper submission, compliance checking, reviewer assignment, and review collection. Our design goals flow directly from that context: the system exists to be *easier to use* than the status quo, *trustworthy* enough for high-stakes academic artifacts, and *open to evolution* as BILSEN's process matures. We ranked every candidate goal against those three forces and kept the three that most shape the day-to-day architecture.

Usability

PRIORITY 1

The core justification for building Bilsen at all is that the current email-plus-Overleaf workflow is slow, error-prone, and opaque. If our system is not dramatically easier to use than that baseline, it fails its purpose. The user base is heterogeneous — authors, reviewers, coordinators, and administrators — with differing levels of tool familiarity, so every routine action (submit a paper, accept an assignment, draft a review, rate a reviewer) must be obvious within seconds of first exposure.

Why we picked it: Direct consequence of the problem statement. Replacing a working-but-painful manual process only pays off when the replacement is visibly less painful. Every coordinator task that still requires leaving the app and falling back to email is a partial failure of the product.

Tradeoff accepted: Rich inline validation, role-specific dashboards, and real-time compliance feedback add frontend complexity and a heavier initial bundle. We accept a slower first paint and a larger UI surface in exchange for fewer support questions, fewer miscoordinated assignments, and fewer reviews lost because a reviewer did not see a notification.

How the design honors it: One home screen per role (My Papers, My Assignments, Admin Dashboard); React Hook Form + Zod for live form validation; inline compliance verdicts shown on the paper detail page; AI-drafted review skeletons so reviewers do not face a blank page; coordinator flows for assignment and extension approval on a single page each.

Reliability

PRIORITY 2

Review artifacts — submitted papers, assignment records, deadline reminders, completed reviews — carry academic weight. A silently lost review, a missed three-day reminder, or a duplicate reviewer assignment can delay a publication or embarrass the coordinator. The system must be predictable and recoverable: every state transition should be durable, every deadline-adjacent event should fire, and every action should be safe to repeat.

Why we picked it: Academic review has low tolerance for silent failure. Users will forgive occasional slowness but not a missing review. Since coordinators cannot easily audit email chains, the system itself is now the authoritative record — that shifts the reliability burden onto us.

Tradeoff accepted: Strong validation, centralized state logic, audit trails, and automated reminders all cost development time and add background work (cron-driven reminders, compliance re-checks, notification retries). We pay that ongoing cost because the alternative — hunting down a lost review after the fact — is

far more expensive.

How the design honors it: All business logic funnels through a single review-service module that owns state transitions for papers, rounds, assignments, reviews, and ratings; every API route validates its input server-side with Zod; a cron endpoint fires deadline reminders at 3-day and 1-day marks plus overdue alerts; persisted JSON collections act as a durable ledger of every change.

Extensibility

PRIORITY 3

BILSEN's review process is not static. New paper categories (tool papers, experience reports, surveys) arrive with their own compliance rules; new venues bring new anonymity requirements; new communication channels (Slack is currently stubbed; Microsoft Teams is plausible) will be demanded; AI features will grow beyond drafting into consensus reports, plagiarism signals, and automated summaries. The architecture must treat these as additions, not rewrites — a new compliance rule or notification channel should be a contained, low-risk change rather than a rippling refactor.

Why we picked it: The product is early in its life and will evolve as BILSEN uses it. A system that forces a rewrite every time a venue changes its page limit or the group adds a Slack integration will collapse under its own maintenance cost. Extensibility is the long-term hedge.

Tradeoff accepted: Designing for extension introduces abstractions — validation profiles, modular API route groups, a pluggable notification dispatcher — that add a layer of indirection and slightly more code than a monolithic direct-call approach. We accept that complexity now because the alternative is either constant rewrites or an accumulation of special cases.

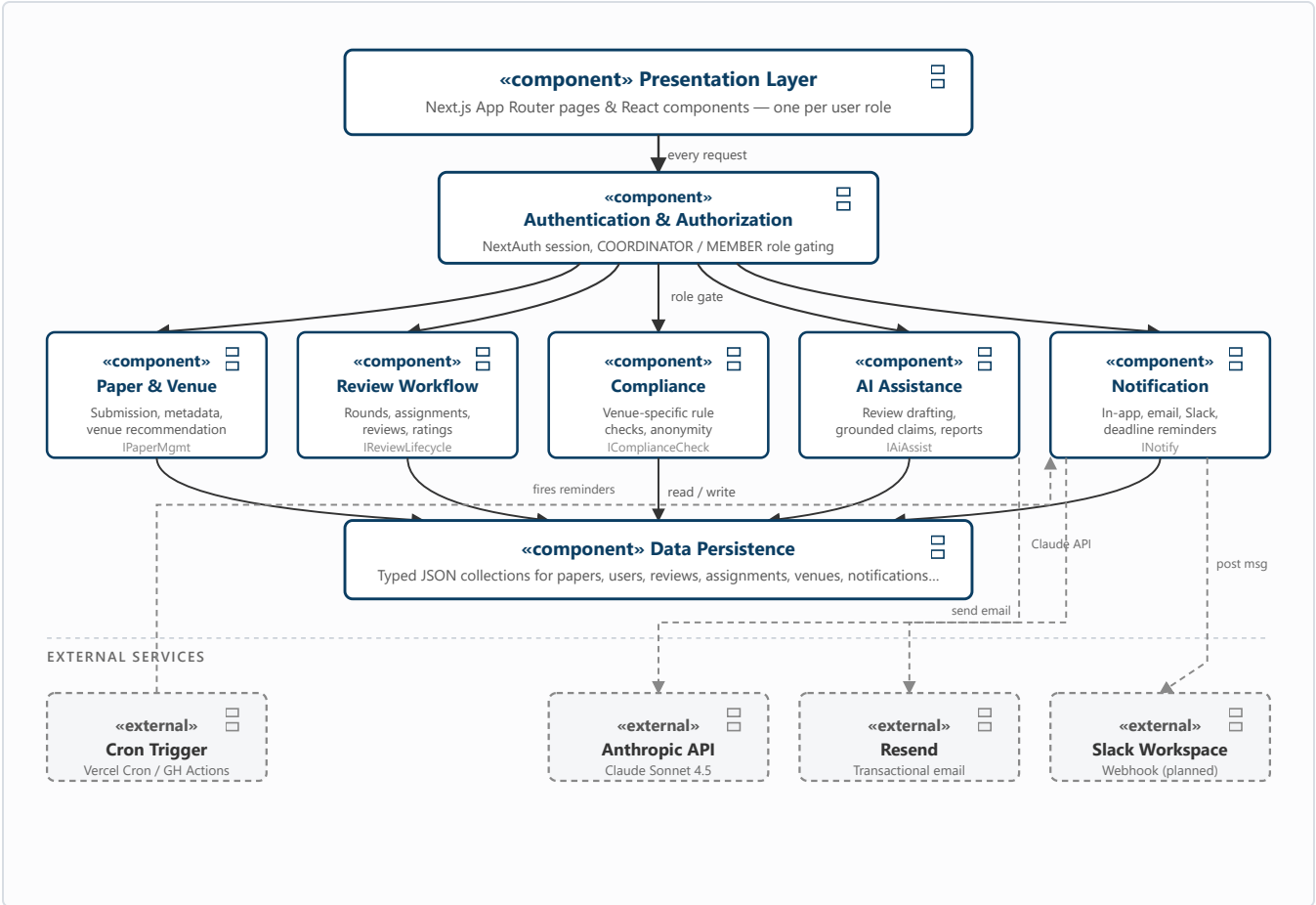
How the design honors it: Compliance rules live in a validation-profiles module that selects rule sets by paper type and track; AI endpoints sit behind a single ai service module so the underlying model can change; notifications flow through a central dispatcher so a new channel (Slack, Teams) is an additive change; API routes are grouped per domain concern so new features attach without touching existing ones.

ALTERNATIVES WE CONSIDERED AND REJECTED

- **Security & Privacy** — genuinely important for double-blind review and confidential unpublished work, but largely satisfied by standard practice (NextAuth sessions, role-gated API routes, automated PDF metadata stripping). We treat it as table-stakes rather than a top-level organizing goal.
- **Performance** — BILSEN is a bounded community of roughly thirty active members. Response times of a few hundred milliseconds are comfortably sufficient; optimizing to sub-50 ms would be over-engineering at this scale.
- **Scalability** — the user base will not grow by an order of magnitude. A single-node deployment with a JSON-backed datastore (with a documented migration path to PostgreSQL) is adequate for the foreseeable horizon.
- **Portability** — the system is a responsive web application, which already runs on any modern browser and device. There is no separate native build to justify portability as a distinct goal.
- **Cost-effectiveness** — infrastructure costs are negligible at this scale (one small cloud instance plus metered AI API usage). Not a meaningful design constraint.
- **Maintainability** — effectively subsumed by Extensibility in our case; we treat it as a subset rather than a separately ranked goal.

2. Subsystem Decomposition

The Bilsen Review System decomposes into eight cohesive subsystems arranged in four architectural tiers (presentation, access control, domain services, persistence) plus a set of external service integrations. The UML component diagram below shows the provided/required interfaces and the direction of dependencies; solid arrows indicate synchronous in-process calls, dashed arrows indicate outbound calls to third-party services or inbound triggers.



□ Internal subsystem («component») □ External service («external») — Solid — synchronous in-process call
---- Dashed — third-party API call or external trigger

SUBSYSTEM	RESPONSIBILITY	KEY CODE
Presentation Layer	Role-specific UI for authors, reviewers, coordinators; renders pages and forwards user actions to the API.	<code>src/app/(dashboard)</code> , <code>src/components</code>
Authentication & AuthZ	Validates credentials, issues sessions, and gates every API route by role (COORDINATOR, MEMBER).	<code>src/auth.ts</code> , <code>proxy.ts</code> , <code>src/lib/users.ts</code>
Paper & Venue	Paper submission, metadata scrubbing, venue CRUD, keyword-based venue recommendation.	<code>src/app/api/papers</code> , <code>src/app/api/venues</code> , <code>src/lib/venue-recommender.ts</code>

SUBSYSTEM	RESPONSIBILITY	KEY CODE
Review Workflow	Creates rounds, assigns reviewers, handles accept/decline/extension, stores submitted reviews and ratings.	<code>src/app/api/rounds,</code> <code>src/app/api/assignments,</code> <code>src/app/api/reviews,</code> <code>src/lib/review-service.ts</code>
Compliance	Runs per-venue rule checks (page limits, sections, reference format, anonymity heuristics, PDF metadata).	<code>src/app/api/compliance,</code> <code>src/lib/validation-profiles.ts</code>
AI Assistance	Claude-powered review drafting, grounded-claim verification, consensus reports, annotated PDF export.	<code>src/app/api/papers/[id]/ai-</code> <code>*,</code> <code>src/lib/ai.ts,</code> <code>src/lib/pdf-report.ts</code>
Notification	In-app notifications, email dispatch via Resend, Slack (stub), deadline reminders triggered by cron.	<code>src/app/api/notifications,</code> <code>src/app/api/cron/reminders,</code> <code>src/lib/email.ts</code>
Data Persistence	Typed JSON collections serving as the authoritative datastore; planned migration path to PostgreSQL.	<code>src/lib/data-store.ts,</code> <code>data/*.json</code>